Министерство образования Новосибирской области
ГБПОУ НСО «Новосибирский авиационный технический колледж имени Б.С. Галущака»

УТВЕРЖДАЮ Председатель цикловой комиссии специальности 09.02.07 _____ А.А. Терехова Протокол №_____ от « __ » _____ 20___ г	УТВЕРЖДАЮ Председатель цикловой комиссии специальности 09.02.07 _____ А.А. Терехова Протокол №_____ от « __ » _____ 20___ г	УТВЕРЖДАЮ Председатель цикловой комиссии специальности 09.02.07 _____ А.А. Терехова Протокол №_____ от « __ » _____ 20___ г

Проектирование и публикация .NET-библиотеки с CI/CD

Методические указания к практическому занятию 3

Профессиональный модуль: ПМ.02 Осуществление интеграции программных модулей для компьютерных систем

Междисциплинарный курс: МДК 02.02 Инструментальные средства разработки программного обеспечения

Специальность: 09.02.07 Информационные системы и программирование

Разработал:
Иванов А.С.

2025

1 Цели

1.1 В ходе выполнения работы студенты осваивают:

1.1.1 Общие компетенции:

– ОК 01 Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;

– ОК 02 Использовать современные средства поиска, анализа и интерпретации информации, и информационные технологии для выполнения задач профессиональной деятельности;

– ОК 09 Пользоваться профессиональной документацией на государственном и иностранном языках.

1.1.2 Профессиональные компетенции:

– ПК 2.3. Выполнять отладку программного модуля с использованием специализированных программных средств.

1.2 В результате выполнения практических заданий студенты:

1.2.1 Получают практический опыт:

– разработки проекта с CI/CD.

1.2.2Осваивают умения:

– использовать выбранную систему контроля версий;;

– использовать методы для получения кода с заданной функциональностью и степенью качества;

– осуществлять разработку кода программного модуля на современных языках программирования.

1.2.3 Усваивают знания:

– основные принципы процесса разработки программного обеспечения;

– основные подходы к интегрированию программных модулей.

2 Оборудование

2.1 Персональный компьютер

2.2 Программное обеспечение (минимальные требования и рекомендуемые варианты):

– Среда разработки Visual Studio 2022 или Visual Studio Code с расширением *C# Dev Kit*;

– Установленный .NET SDK (версия 8.0 или выше);

– Git — клиентская утилита Git (>= 2.30).

3 Форма организации:индивидуальная

4 Инструктаж

4.1 Организация рабочего процесса перед началом работы:

– проверить готовность рабочего места: наличие ПК, доступ в Интернет, установленный .NET SDK и Visual Studio / Visual Studio Code;

– убедиться, что установлен Git и выполнен вход в GitHub (наличие учётной записи).

4.2 Время выполнения работы – 90 минут.

5 Порядок выполнения

5.1 Ознакомиться с содержанием задания

5.2 Ознакомиться с методическими рекомендациями по выполнению работы

5.3 Выполнить задание практической работы

5.4 Сформировать отчет по проделанной работе (см. п. 7 Форма отчета)

6 Методические рекомендации

Методические рекомендации представлены в приложении от А до Б.

7 Форма отчета

7.1 Номер и название практического занятия

7.2 Цели практического занятия

7.3 Ход выполнения работы:

– структуру проекта,

– код библиотеки и тестов,

– конфигурацию workflows,

– ссылку на опубликованный пакет в GitHub Packages.

7.4 Краткий вывод по проделанной работе

8 Содержание задания

1. Создать новый репозиторий на GitHub (Не забывайте выполнять commit, push, Pull request);

2. Организовать структуру решения в Visual Studio/VS Code:

– библиотека Todo.Core (классическая .NET Class Library),

– проект с тестами Todo.Core.Tests (xUnit);

3. Реализовать простую библиотеку (например, базовые операции со списком задач — добавить, удалить, отметить выполненной);

4. Написать unit-тесты для библиотеки (xUnit);

5. Настроить workflow ci.yml для автоматической сборки и тестирования:

– checkout кода,

– установка .NET SDK,

– restore, build, test, pack;

6. Настроить workflow publish-package.yml для публикации пакета в GitHub Packages (Maven/NuGet):

– запуск после успешного CI,

– упаковка .nupkg,

– публикация с использованием secrets.GITHUB_TOKEN;

7. Подготовить README.md с инструкцией по подключению пакета через NuGet;

8. Проверить прохождение Actions на GitHub;

9. Реализовать самостоятельное задание.

9 Критерии оценок

9.1 При оценивании результатов выполнения заданий учитываются следующие критерии:

– практическое задание выполнено в соответствии с методическими рекомендациями, в установленное время (обязательный, на оценку «отлично» и «хорошо»);

– структура решения: есть библиотека и отдельный проект для тестов;

– код оформлен по соглашениям C# (PascalCase, CamelCase, понятные имена методов и переменных);

– реализованы unit-тесты и они проходят успешно;

– настроен CI (build + test + pack);

– настроен CD (публикация в GitHub Packages);

– README.md оформлен и содержит инструкцию по установке пакета и использованию библиотеки;

– отчет оформлен в соответствии с п.п.7 (обязательный на оценку «отлично» и «хорошо»);

9.2 В основу оценки выполненных заданий положен принцип:

– «отлично» – при условии выполнения от семи до восьми критериев, в соответствии с п. 9.1, и сдачи в заданные сроки;

– «хорошо» – при условии выполнения от пяти до шести критериев в соответствии с п. 9.1, и сдачи в заданные сроки;

– «удовлетворительно» – при условии выполнения минимум от трех до четырёх критериев в соответствии с 9.1;

– «неудовлетворительно» – практическое задание не выполнено или выполнено менее трех критериев, в соответствии с п. 9.1.

Приложение А

(справочное)

NuGet-пакет vs обычная сборка (DLL)

DLL — это один файл сборки (.dll). Содержит ваш скомпилированный код.

NuGet-пакет (.nupkg) — это архив (zip) с метаданными и одним или несколькими артефактами (DLL, symbols, readme). Пакет предназначен для распространения и управления зависимостями между проектами.

Таблица сравнения

Характеристика	Обычная сборка (DLL)	NuGet-пакет (.nupkg)
Содержимое	Один или несколько .dll и ресурсы	.dll + метаданные (PackageId, Version, Description) + опционально symbols/readme
Распространение	Передаётся вручную (копирование / подключение)	Через репозиторий (nuget.org, GitHub Packages, приватный feed)
Управление зависимостями	Ручное (нужно вручную положить все зависимости в папку)	Автоматическое (NuGet подтягивает транзитивные зависимости)
Версионирование	Управляется вручную (файлы с разными именами/папками)	SemVer — версия пакета в метаданных → легко управлять версиями
Удобство использования	Подходит для простых локальных сценариев	Подходит для библиотек, распространяемых между проектами/командами
Публикация через CI/CD	Копирование в артефакт хранилище / релиз	`dotnet pack` + `dotnet nuget push` → репозиторий пакетов
Поддержка зависимостей	Нет транзитивности	Да, NuGet управляет транзитивными зависимостями
Подпись и symbols	Можно подписать сборку, хранить pdb отдельно	Можно публиковать symbol package и source link

Когда что использовать (правило выбора)

Используйте DLL:

– при локальной проверке/прототипировании;

– если нужно быстро подключить одну сборку в один проект (малый scope);

– для внутренних артефактов без необходимости версионирования и распространения.

Используйте NuGet-пакет:

– если библиотека будет использоваться в нескольких проектах/репозиториях;

– если нужна явная версия и возможность откатиться/обновиться;

– хочется автоматизировать доставку через CI/CD (pack → push → install).

Приложение Б

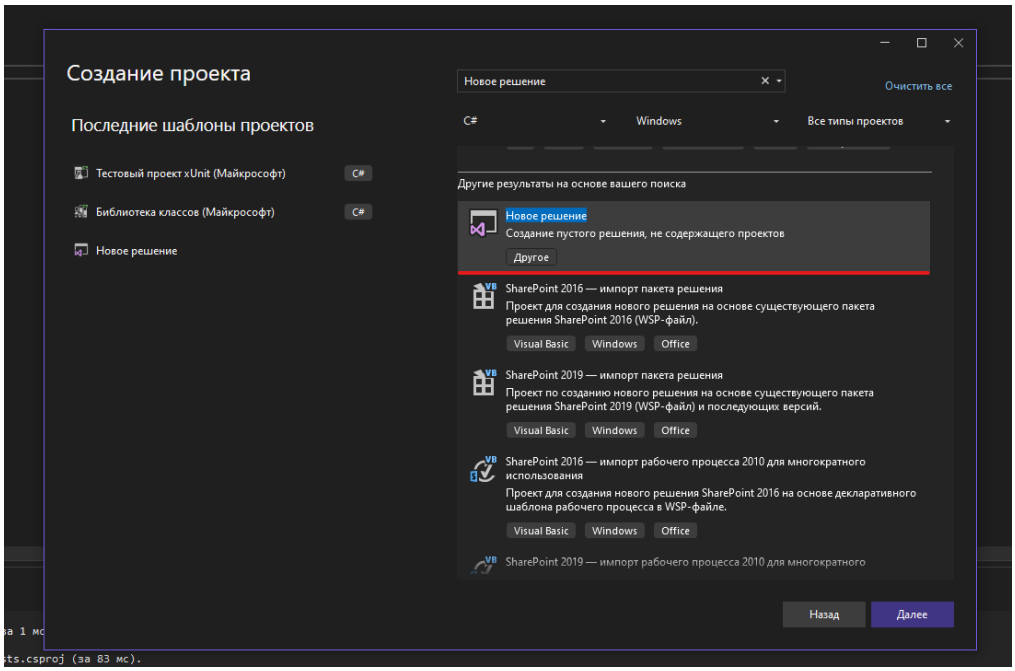
(справочное)

Пример реализации

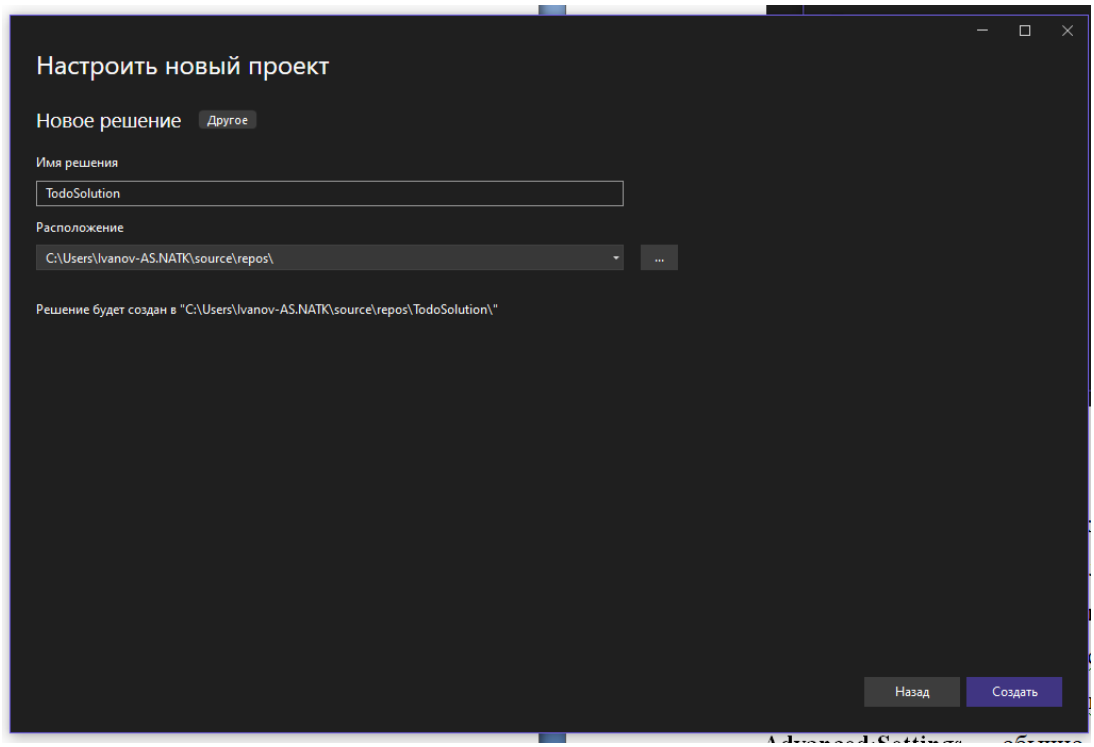
Здесь показана реализация без работы с git`ом!!!

1. Настройка проекта

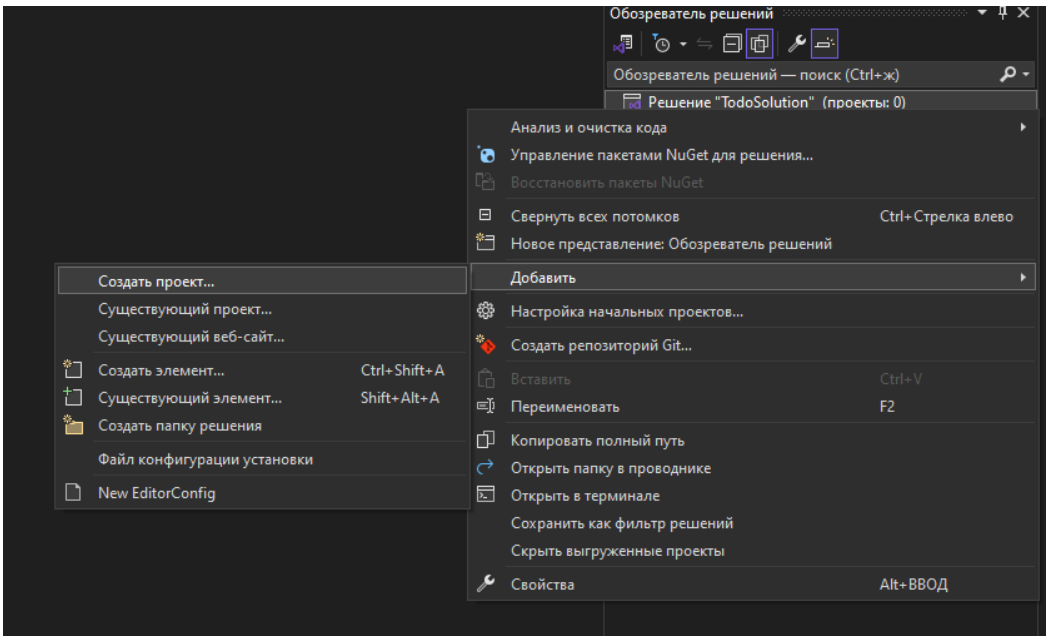
Создайте проект с шаблоном «Новое решение»:



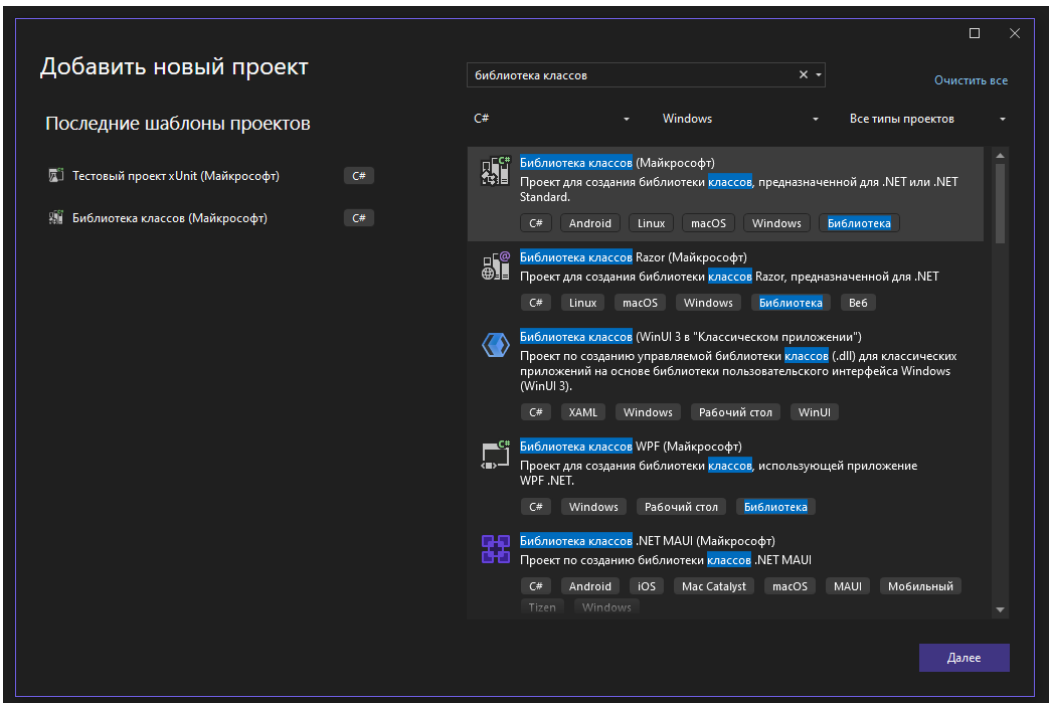
Имя решения: TodoSolution.



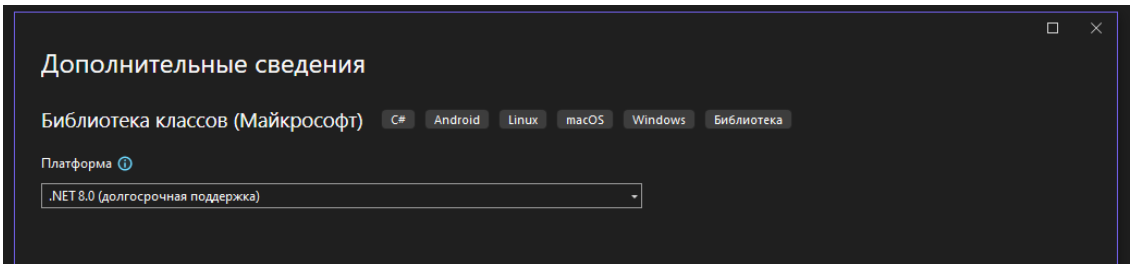
Добавить проект библиотеки (Todo.Core). Для этого:



Выберите шаблон «Библиотека классов (Майкрософт)»:

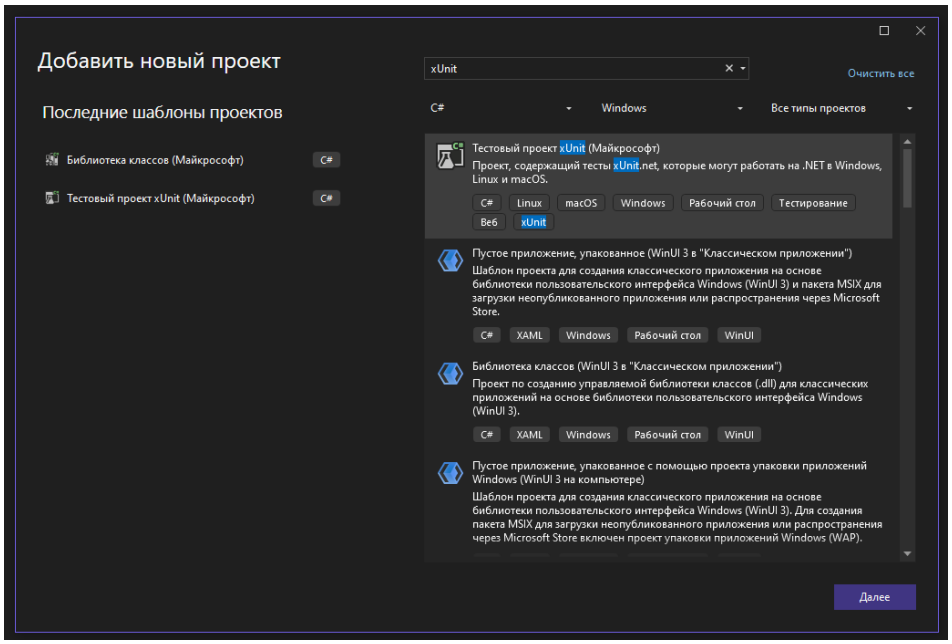


В качестве платформы выберите .NET 8.0



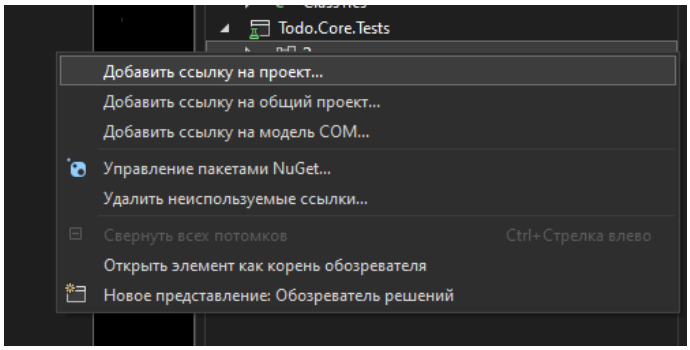
Имя проекта: Todo.Core

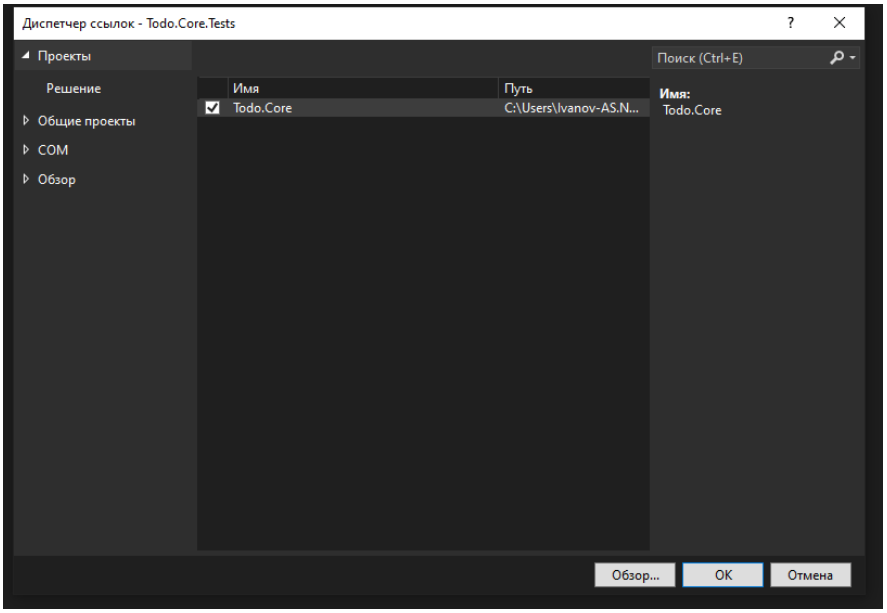
Добавить тестовый проект (xUnit). Для этого Выберите шаблон **xUnit Test Project** (Майкрософт):



Платформа также .NET 8.0 и имя проекта: Todo.Core.Tests

После чего добавьте для тестового проекта ссылку на библиотеку Todo.Core:





В Todo.Core добавьте классы TodoItem и TodoList.

Пример реализации TodoItem:

```
namespace Todo.Core
{
    public class TodoItem
    {
        public Guid Id { get; } = Guid.NewGuid();
        public string Title { get; private set; }
        public bool IsDone { get; private set; }

        public TodoItem(string title)
        {
            Title = title?.Trim() ?? throw new ArgumentNullException(nameof(title));
        }

        public void MarkDone() => IsDone = true;
        public void MarkUndone() => IsDone = false;
        public void Rename(string newTitle)
        {
            if (string.IsNullOrWhiteSpace(newTitle)) throw new ArgumentException("Title
required", nameof(newTitle));
            Title = newTitle.Trim();
        }
    }
}
```

Пример реализации TodoList:

```
using System.Collections.Generic;
using System.Linq;

namespace Todo.Core
{
    public class TodoList
    {
        private readonly List<TodoItem> _items = new();

        public IReadOnlyList<TodoItem> Items => _items.AsReadOnly();

        public TodoItem Add(string title)
        {
            var item = new TodoItem(title);
            _items.Add(item);
            return item;
        }

        public bool Remove(Guid id) => _items.RemoveAll(i => i.Id == id) > 0;

        public IEnumerable<TodoItem> Find(string substring) =>
            _items.Where(i => i.Title.Contains(substring ?? string.Empty,
StringComparison.OrdinalIgnoreCase));

        public int Count => _items.Count;
    }
}
```

В Todo.Core.Tests добавьте класс TodoListTests. Пример реализации:

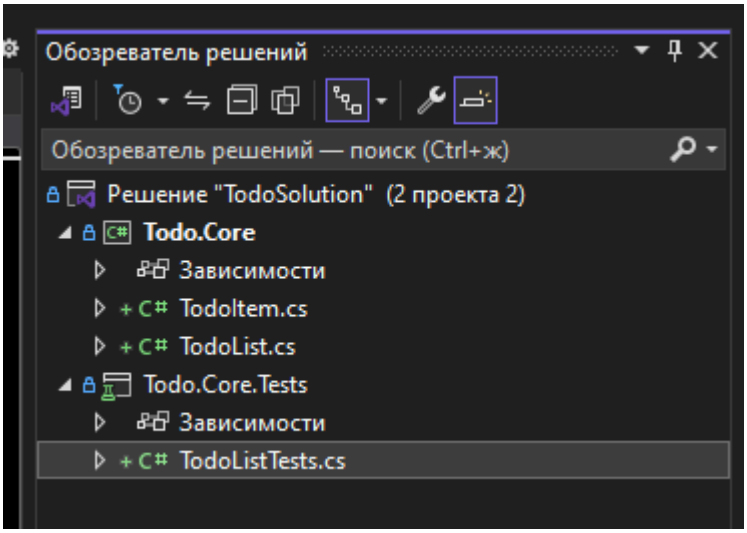
```
using Xunit;
using Todo.Core;
using System.Linq;

namespace Todo.Core.Tests
{
    public class TodoListTests
    {
        [Fact]
        public void Add_IncreasesCount()
        {
            var list = new TodoList();
            list.Add("  task  ");
            Assert.Equal(1, list.Count);
            Assert.Equal("task", list.Items.First().Title);
        }

        [Fact]
        public void Remove_ById_Works()
        {
            var list = new TodoList();
            var item = list.Add("a");
            Assert.True(list.Remove(item.Id));
            Assert.Equal(0, list.Count);
        }

        [Fact]
        public void Find_ReturnsMatches()
        {
            var list = new TodoList();
            list.Add("Buy milk");
            list.Add("Read book");
            var found = list.Find("buy").ToList();
            Assert.Single(found);
            Assert.Equal("Buy milk", found[0].Title);
        }
    }
}
```

Пример структуры проекта:



Нажмите на Todo.Core и допишите следующую реализацию:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
     <GeneratePackageOnBuild>false</GeneratePackageOnBuild>
     <PackageId>Todo.Core</PackageId>
     <Version>0.1.0</Version>
     <Authors>Alexiv</Authors>
     <Company>io.natk</Company>
     <Description>Simple Todo core library (for practice)</Description>
     <PackageRequireLicenseAcceptance>false</PackageRequireLicenseAcceptance>
  </PropertyGroup>

</Project>
```

<TargetFramework>net8.0</TargetFramework>:

Определяет целевую платформу сборки. Важно: если версия SDK в раннере отличается (например, устанавливается 7.0), сборка может упасть. В workflow указывайте ту же версию.

<ImplicitUsings>enable</ImplicitUsings> и <Nullable>enable</Nullable>:

Упрощают код (авто using) и включают проверку nullable-аннотаций. Это не влияет на CI/CD напрямую, но улучшает качество кода и помогает ловить ошибки на этапе компиляции в CI.

<GeneratePackageOnBuild>false</GeneratePackageOnBuild>:

– Ключевой флаг для CI: при false dotnet build **не будет** автоматически генерировать .nupkg.

– Почему false удобно для CI: вы явно контролируете упаковку через dotnet pack в отдельном шаге, указываете выходную папку (-o ./artifacts), версию и параметры, проще отлавливать и собирать артефакты.

– Если поставить true, то .nupkg генерируется на каждом локальном билде — удобно локально, но в CI обычно делают pack явным.

<PackageId>Todo.Core</PackageId>:

Идентификатор пакета в NuGet/GitHub Packages. Используется при публикации и при установке пакета в другие проекты.

<Version>0.1.0</Version>:

Версия пакета. CI может читать её для создания тега/релиза или переопределять при сборке (dotnet pack -p:Version=0.1.1).

<Authors>, <Company>, <Description>, <PackageRequireLicenseAcceptance>false</PackageRequireLicenseAcceptance>:

Описательные метаданные, попадают в .nupkg и на страницу пакета. PackageRequireLicenseAcceptance = false означает, что потребителям не нужно вручную соглашаться с лицензией при установке пакета — удобнее для учебных пакетов.

Создайте файл .github/workflows/ci.yml (Visual Studio не создаёт эти файлы автоматически). Пример реализации:

```yaml
name: CI / Build & Test

on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v3
        with:
          dotnet-version: '8.0.x'
      - name: Cache NuGet packages
        uses: actions/cache@v4
        with:
          path: ~/.nuget/packages
          key: ${{ runner.os }}-nuget-${{ hashFiles('**/*.csproj', '**/packages.lock.json') }}
          restore-keys: |
            ${{ runner.os }}-nuget-

      - name: Restore
        run: dotnet restore

      - name: Build
        run: dotnet build --no-restore -c Release

      - name: Test
        run: dotnet test --no-build -c Release --verbosity normal

      - name: Pack
        run: dotnet pack Todo.Core/Todo.Core.csproj -c Release -o ./artifacts
```

```yaml
- name: Setup .NET
  uses: actions/setup-dotnet@v3
  with:
    dotnet-version: '8.0.x'
```

Сборка и тесты требуют установленного SDK той же версии, что и в проекте (TargetFramework в .csproj). Поэтому здесь устанавливаем required .NET SDK (8.x).

```
- name: Cache NuGet packages
  uses: actions/cache@v4
  with:
    path: ~/.nuget/packages
    key: ${{ runner.os }}-nuget-${{ hashFiles('**/*.csproj', '**/packages.lock.json')
}}
    restore-keys: |
      ${{ runner.os }}-nuget-
```

Кеширует локальный NuGet-кэш (~/.nuget/packages) между прогономами для ускорения dotnet restore.

```
- name: Restore
  run: dotnet restore
```

Скачивает NuGet-зависимости

```
- name: Build
  run: dotnet build --no-restore -c Release
```

Компилирует решение/проекты в режиме Release, без повторного восстановления пакетов (параметр --no-restore — restore уже выполнён). Это необходимо, чтобы проверить, что проект собирается, и сгенерировать артефакты для тестов/пакета.

```
- name: Test
  run: dotnet test --no-build -c Release --verbosity normal
```

Запускает тесты (xUnit/NUnit/MSTest) без повторной сборки (--no-build). --verbosity normal — вывод средней детализации.

```
- name: Pack
  run: dotnet pack Todo.Core/Todo.Core.csproj -c Release -o ./artifacts
```

Создаёт NuGet-пакет (.nupkg) для проекта Todo.Core в Release конфигурации и складывает его в папку ./artifacts

Создайте .github/workflows/publish-package.yml. Пример реализации:

```yaml
name: Publish package to GitHub Packages

on:
  workflow_run:
    workflows: ["CI / Build & Test"]
    types:
      - completed

jobs:
  publish:
    if: ${{ github.event.workflow_run.conclusion == 'success' }}
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v3
        with:
          dotnet-version: '8.0.x'

      - name: Restore
        run: dotnet restore

      - name: Build (Release)
        run: dotnet build -c Release --no-restore

      - name: Pack
        run: dotnet pack Todo.Core/Todo.Core.csproj -c Release -o ./artifacts

      - name: List artifacts
        run: ls -la ./artifacts || true

      - name: Push package to GitHub Packages (NuGet)
        run: dotnet nuget push ./artifacts/*.nupkg --source "https://nuget.pkg.github.com/${{ github.repository_owner }}/index.json"
--api-key ${{ secrets.GITHUB_TOKEN }} --skip-duplicate
```

```yaml
on:
  workflow_run:
    workflows: ["CI / Build & Test"]
    types:
      - completed
```

Воркфлоу запускается **после завершения** указанного workflow (CI / Build & Test)

```yaml
jobs:
  publish:
    if: ${{ github.event.workflow_run.conclusion == 'success' }}
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
```

Проверяет, что предыдущий workflow завершился успешно. Если упал CI — CD не запустится.

runs-on: указывает на использование Linux runner (ubuntu-latest).

permissions:

– contents: read — нужно для доступа к исходникам.

– packages: write — разрешение на публикацию пакетов в GitHub Packages.

```
- name: Build (Release)
  run: dotnet build -c Release --no-restore
```

Компилирует проект в конфигурации Release (без повторного restore).

```
- name: Pack
  run: dotnet pack Todo.Core/Todo.Core.csproj -c Release -o ./artifacts
```

Собирает **NuGet-пакет (.nupkg)** из проекта Todo.Core в папку ./artifacts.

```
- name: List artifacts
  run: ls -la ./artifacts || true
```

Отладочный шаг — проверка, что .nupkg реально собрался.

```
- name: Push package to GitHub Packages (NuGet)
  run: dotnet nuget push ./artifacts/*.nupkg --source "https://nuget.pkg.github.com/${{ github.repository_owner }}/index.json" --api-key ${{ secrets.GITHUB_TOKEN }} --skip-duplicate
```

отправляет .nupkg в GitHub Packages.

– ./artifacts/*.nupkg — путь к пакету.

– --source "https://nuget.pkg.github.com/${{ github.repository_owner }}/index.json" — URL GitHub Packages для текущего репозитория.

– --api-key ${{ secrets.GITHUB_TOKEN }} — ключ доступа (по умолчанию GitHub Actions выдаёт токен).

– --skip-duplicate — если версия пакета уже опубликована, ошибка не произойдёт, шаг просто пропустится.

Как подключить опубликованный пакет в другом проекте в Visual Studio

1. В Visual Studio для другого проекта **Tools → Options → NuGet Package Manager → Package Sources** добавьте GitHub Packages URL: https://nuget.pkg.github.com/<OWNER>/index.json.

2. При попытке установить пакет NuGet спросит учетные данные. Для приватного пакета укажите username = GITHUB_ACTOR (ваш логин) и password = PAT (или Personal Access Token).

3. В Package Manager → Browse выберите источник GitHubPackages и установите пакет.

Для публичных пакетов аутентификация не нужна.

Задание для самостоятельной реализации. JSON-персистенция задач:

Реализовать сохранение/загрузгу списка задач в/из JSON-файла.

Для этого: добавить методы Save(string path) и Load(string path) в TodoList с использованием System.Text.Json или Newtonsoft.Json.

Команды: тесты: dotnet test

Подсказка: сериализуйте IReadOnlyList<TodoItem>; в TodoItem пометьте свойства { get; private set; } как сериализуемые.

Результат: тесты на сохранение/загрузку